SQL Server Functions

A Beginner's Guide for Students

String | Numeric | Date | Conversion | NULL | Aggregate | Window

How to use this guide
Each section has a short explanation, a table of functions with examples, and practice questions at the end. Type every example in SSMS (SQL Server Management Studio) and observe the output. Learning SQL is 20% reading and 80% practice.

Contents

1. What is a function?
2. String functions
3. Numeric (math) functions
4. Date and time functions
5. Conversion functions
6. NULL handling and logical functions
7. Aggregate functions
8. Ranking (window) functions
9. Analytic functions: LAG, LEAD and more
10. Other useful functions
11. Practice exercises and answers

1. What is a function?

A **function** is a ready-made piece of SQL that takes an input, does some work, and returns a value. You use functions inside `SELECT`, `WHERE`, `ORDER BY` and more.

Two big families:
• **Scalar functions**: work on one value (or one row) and return one value. Example: `UPPER('sql')` returns SQL.
• **Aggregate functions**: work on many rows and return one value. Example: `SUM(salary)`.

Sample table used in this guide: Employees

EmpID	Name	Dept	Salary	JoinDate
1	Ravi	IT	50000	2021-03-15
2	Sara	HR	42000	2020-07-01
3	John	IT	60000	2019-11-20
4	Meena	Sales	38000	2022-01-10
5	Arun	IT	50000	2023-05-05

```
SELECT UPPER(Name) AS NameInCaps, Salary
FROM Employees;
```

2. String functions

Used to clean, search, cut, join and reshape text.

Function	What it does	Example	Result
`LEN(s)`	Number of characters (ignores trailing spaces)	`SELECT LEN('Hello')`	5
`DATALENGTH(s)`	Bytes used (includes trailing spaces)	`SELECT DATALENGTH('Hi ')`	3
`UPPER(s)`	Converts to capital letters	`SELECT UPPER('sql')`	SQL
`LOWER(s)`	Converts to small letters	`SELECT LOWER('SQL')`	sql
`LEFT(s,n)`	First n characters	`SELECT LEFT('Database',4)`	Data
`RIGHT(s,n)`	Last n characters	`SELECT RIGHT('Database',4)`	base
`SUBSTRING(s,start,len)`	Cuts part of a string (start from 1)	`SELECT SUBSTRING('Database',5,4)`	base
`CHARINDEX(find,s)`	Position of text inside string (0 if not found)	`SELECT CHARINDEX('b','Database')`	5
`PATINDEX('%p%',s)`	Like CHARINDEX but supports wildcards	`SELECT PATINDEX('%ba%','Database')`	5
`REPLACE(s,old,new)`	Replaces all matches	`SELECT REPLACE('a-b-c','-','/')`	a/b/c
`STUFF(s,start,len,new)`	Deletes part and inserts new text	`SELECT STUFF('Hello',1,1,'J')`	Jello
`LTRIM(s)`	Removes leading spaces	`SELECT LTRIM(' hi')`	hi
`RTRIM(s)`	Removes trailing spaces	`SELECT RTRIM('hi ')`	hi
`TRIM(s)`	Removes both-side spaces (2017+)	`SELECT TRIM(' hi ')`	hi
`CONCAT(a,b,...)`	Joins values; treats NULL as empty	`SELECT CONCAT('Sql',' ','Server')`	Sql Server
`CONCAT_WS(sep,a,b,...)`	Joins with a separator (2017+)	`SELECT CONCAT_WS('-','2024','05','01')`	2024-05-01
`REVERSE(s)`	Reverses the text	`SELECT REVERSE('abc')`	cba
`REPLICATE(s,n)`	Repeats text n times	`SELECT REPLICATE('ab',3)`	ababab
`SPACE(n)`	Returns n spaces	`SELECT 'A'+SPACE(2)+'B'`	A B
`FORMAT(v,fmt)`	Formats numbers/dates as text	`SELECT FORMAT(1234.5,'N2')`	1,234.50
`STRING_AGG(col,sep)`	Joins rows into one string (2017+)	`SELECT STRING_AGG(Name,', ') FROM Employees`	Ravi, Sara, ...
`STRING_SPLIT(s,sep)`	Splits string into rows (2016+)	`SELECT value FROM STRING_SPLIT('a,b,c',',')`	a / b / c (3 rows)
`ASCII(c) / CHAR(n)`	Character to code and back	`SELECT ASCII('A'), CHAR(66)`	65, B
`UNICODE(c) / NCHAR(n)`	Unicode version of ASCII/CHAR	`SELECT UNICODE('A')`	65
`SOUNDEX(s)`	Sound-alike code for names	`SELECT SOUNDEX('Smith')`	S530
`DIFFERENCE(a,b)`	Similarity of SOUNDEX codes (0 to 4)	`SELECT DIFFERENCE('Smith','Smyth')`	4

Function	What it does	Example	Result
`QUOTENAME(s)`	Wraps name in [ ]	`SELECT QUOTENAME('Order Details')`	[Order Details]
`TRANSLATE(s,from,to)`	Character-by-character replace (2017+)	`SELECT TRANSLATE('2*3','*','x')`	2x3

Real-life example: first name from a full name

```
SELECT LEFT('Ravi Kumar', CHARINDEX(' ','Ravi Kumar') - 1) AS FirstName;
-- Result: Ravi
```

Tip: + returns NULL if any part is NULL; CONCAT does not. Prefer CONCAT for safety.

3. Numeric (math) functions

Used for rounding, powers, signs and random values.

Function	What it does	Example	Result
`ABS(n)`	Removes the minus sign	`SELECT ABS(-15)`	15
`CEILING(n)`	Rounds up to whole number	`SELECT CEILING(4.2)`	5
`FLOOR(n)`	Rounds down to whole number	`SELECT FLOOR(4.9)`	4
`ROUND(n,d)`	Rounds to d decimals	`SELECT ROUND(123.456,2)`	123.460
`ROUND(n,-d)`	Rounds to the left of the point	`SELECT ROUND(1234,-2)`	1200
`POWER(n,p)`	n raised to p	`SELECT POWER(2,3)`	8
`SQRT(n)`	Square root	`SELECT SQRT(16)`	4
`SQUARE(n)`	n times n	`SELECT SQUARE(5)`	25
`SIGN(n)`	-1, 0 or 1 by sign	`SELECT SIGN(-9)`	-1
`n % m`	Remainder (modulo)	`SELECT 10 % 3`	1
`EXP(n) / LOG(n)`	e^n and natural log	`SELECT LOG(EXP(1))`	1
`LOG10(n)`	Log base 10	`SELECT LOG10(1000)`	3
`PI()`	Value of pi	`SELECT PI()`	3.14159...
`RAND()`	Random float between 0 and 1	`SELECT RAND()`	0.7134...
`RAND(seed)`	Repeatable random value	`SELECT RAND(5)`	same each time
`NEWID()`	Random unique identifier (GUID)	`SELECT NEWID()`	e.g. 6F9619FF-...
`TRUNCATE style`	Use ROUND(n,d,1) to cut instead of round	`SELECT ROUND(5.678,1,1)`	5.600
`SIN, COS, TAN`	Trigonometry (radians)	`SELECT COS(0)`	1

Real-life example: random 3 employees

```
SELECT TOP 3 * FROM Employees ORDER BY NEWID();
```

4. Date and time functions

Dates are the most used data type in business reports. Learn these well.

Common datepart values: year (yy), quarter (qq), month (mm), day (dd), week (wk), hour (hh), minute (mi), second (ss).

Function	What it does	Example	Result
`GETDATE()`	Current date and time of server	`SELECT GETDATE()`	2026-10-26 14:30:00
`SYSDATETIME()`	Current date/time, higher precision	`SELECT SYSDATETIME()`	2026-10-26 14:30:00.1234567
`GETUTCDATE()`	Current UTC date/time	`SELECT GETUTCDATE()`	UTC time
`CAST(GETDATE() AS DATE)`	Today's date only	`SELECT CAST(GETDATE() AS DATE)`	2026-10-26
`YEAR(d)`	Year part	`SELECT YEAR('2024-05-17')`	2024
`MONTH(d)`	Month number	`SELECT MONTH('2024-05-17')`	5
`DAY(d)`	Day of month	`SELECT DAY('2024-05-17')`	17
`DATEPART(part,d)`	Any part as a number	`SELECT DATEPART(quarter,'2024-05-17')`	2
`DATENAME(part,d)`	Any part as text	`SELECT DATENAME(month,'2024-05-17')`	May
`DATEADD(part,n,d)`	Adds (or subtracts with -n)	`SELECT DATEADD(day,10,'2024-05-17')`	2024-05-27
`DATEDIFF(part,d1,d2)`	Number of boundaries crossed	`SELECT DATEDIFF(day,'2024-01-01','2024-03-01')`	60
`EOMONTH(d)`	Last day of the month	`SELECT EOMONTH('2024-02-10')`	2024-02-29
`EOMONTH(d,n)`	Last day n months away	`SELECT EOMONTH('2024-02-10',1)`	2024-03-31
`DATEFROMPARTS(y,m,d)`	Builds a date	`SELECT DATEFROMPARTS(2024,12,25)`	2024-12-25
`ISDATE(s)`	1 if valid date, else 0	`SELECT ISDATE('2024-13-45')`	0
`FORMAT(d,fmt)`	Date as formatted text	`SELECT FORMAT('2024-05-17','dd-MMM-yyyy')`	17-May-2024
`DATETRUNC(part,d)`	Cuts date down to part (2022+)	`SELECT DATETRUNC(month,'2024-05-17')`	2024-05-01
`DATE_BUCKET(...)`	Groups dates into buckets (2022+)	`(advanced)`	-
`SWITCHOFFSET / AT TIME ZONE`	Time zone conversion	`SELECT SYSDATETIMEOFFSET()`	date with offset

Real-life examples

```
-- Age in years (approximate)
SELECT Name, DATEDIFF(year, JoinDate, GETDATE()) AS YearsInCompany
FROM Employees;

-- First day of the current month
SELECT DATEFROMPARTS(YEAR(GETDATE()), MONTH(GETDATE()), 1);

-- Employees who joined in 2021
SELECT * FROM Employees WHERE YEAR(JoinDate) = 2021;
```

Beginner warning: `DATEDIFF` counts boundaries crossed, not full periods. `DATEDIFF(year,'2023-12-31','2024-01-01')` returns 1 even though only one day passed.

5. Conversion functions

SQL Server sometimes converts types automatically (implicit), but you should convert explicitly to avoid surprises.

Function	What it does	Example	Result
`CAST(x AS type)`	Standard ANSI conversion	`SELECT CAST('123' AS INT)`	123
`CONVERT(type,x,style)`	SQL Server conversion with styles	`SELECT CONVERT(VARCHAR,GETDATE(),103)`	26/10/2026
`TRY_CAST(x AS type)`	Returns NULL instead of error	`SELECT TRY_CAST('abc' AS INT)`	NULL
`TRY_CONVERT(type,x)`	Safe version of CONVERT	`SELECT TRY_CONVERT(DATE,'2024-99-99')`	NULL
`PARSE(s AS type)`	Culture-aware text to type	`SELECT PARSE('5 Jan 2024' AS DATE)`	2024-01-05
`TRY_PARSE(s AS type)`	Safe version of PARSE	`SELECT TRY_PARSE('xyz' AS DATE)`	NULL
`STR(n,len,dec)`	Number to text with padding	`SELECT STR(3.14159,6,2)`	3.14

Popular CONVERT date styles

Function	What it does	Example	Result
`Style 101`	US format mm/dd/yyyy	`SELECT CONVERT(VARCHAR,GETDATE(),101)`	10/26/2026
`Style 103`	British/Indian dd/mm/yyyy	`SELECT CONVERT(VARCHAR,GETDATE(),103)`	26/10/2026
`Style 105`	Italian dd-mm-yyyy	`SELECT CONVERT(VARCHAR,GETDATE(),105)`	26-10-2026
`Style 112`	ISO yyyymmdd	`SELECT CONVERT(VARCHAR,GETDATE(),112)`	20261026
`Style 108`	Time hh:mi:ss	`SELECT CONVERT(VARCHAR,GETDATE(),108)`	14:30:00

Tip: When loading data from files, use TRY_CAST so one bad row does not crash the whole query.

6. NULL handling and logical functions

NULL means unknown or missing. It is not zero and not an empty string. Any math with NULL gives NULL, and `NULL = NULL` is not true. Use `IS NULL` to test it.

Function	What it does	Example	Result
`ISNULL(x,repl)`	Replaces NULL with a value (2 arguments)	`SELECT ISNULL(NULL,0)`	0
`COALESCE(a,b,c,...)`	First non-NULL value (ANSI, many args)	`SELECT COALESCE(NULL,NULL,5)`	5
`NULLIF(a,b)`	Returns NULL if a = b, else a	`SELECT 10 / NULLIF(0,0)`	NULL (no error)
`IIF(test,yes,no)`	Short IF/ELSE (2012+)	`SELECT IIF(5>3,'Big','Small')`	Big
`CHOOSE(i,a,b,c)`	Picks item by position	`SELECT CHOOSE(2,'A','B','C')`	B
`CASE WHEN ... END`	Full if-else-if logic	`see below`	-
`GREATEST(a,b,...)`	Largest value (2022+)	`SELECT GREATEST(3,9,5)`	9
`LEAST(a,b,...)`	Smallest value (2022+)	`SELECT LEAST(3,9,5)`	3

CASE example: salary grade

```
SELECT Name, Salary,
  CASE
    WHEN Salary >= 55000 THEN 'High'
    WHEN Salary >= 40000 THEN 'Medium'
    ELSE 'Low'
  END AS Grade
FROM Employees;
```

ISNULL vs COALESCE: ISNULL is SQL Server only and takes exactly 2 arguments. COALESCE is standard SQL and accepts many. Use COALESCE for portable code.

7. Aggregate functions

Aggregates summarize many rows into one result. They ignore NULLs (except `COUNT(*)`). Combine with `GROUP BY` for per-group results.

Function	What it does	Example	Result
`COUNT(*)`	Counts all rows	`SELECT COUNT(*) FROM Employees`	5
`COUNT(col)`	Counts non-NULL values	`SELECT COUNT(Dept) FROM Employees`	5
`COUNT(DISTINCT col)`	Counts unique values	`SELECT COUNT(DISTINCT Dept) FROM Employees`	3
`SUM(col)`	Total	`SELECT SUM(Salary) FROM Employees`	240000
`AVG(col)`	Average	`SELECT AVG(Salary) FROM Employees`	48000
`MIN(col)`	Smallest value	`SELECT MIN(Salary) FROM Employees`	38000
`MAX(col)`	Largest value	`SELECT MAX(Salary) FROM Employees`	60000
`STDEV / VAR`	Standard deviation / variance	`SELECT STDEV(Salary) FROM Employees`	statistical
`STRING_AGG`	Joins values into one text	`SELECT STRING_AGG(Name,',') FROM Employees`	Ravi,Sara,...
`COUNT_BIG(*)`	COUNT for very large tables	`SELECT COUNT_BIG(*) FROM Employees`	5
`APPROX_COUNT_DISTINCT`	Fast estimate of unique count	`(big data)`	-

GROUP BY and HAVING

```
SELECT Dept, COUNT(*) AS Total, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Dept
HAVING COUNT(*) > 1;      -- filters groups, not rows
```

Remember: `WHERE` filters rows *before* grouping. `HAVING` filters groups *after* grouping.

Extra grouping helpers

```
GROUP BY ROLLUP (Dept)     -- adds a grand total row
GROUP BY CUBE (Dept, Year) -- totals for every combination
GROUPING SETS ((Dept),())  -- you choose the combinations
```

8. Ranking (window) functions

Window functions calculate across related rows **without collapsing them**. They use `OVER (PARTITION BY ... ORDER BY ...)`.

Function	What it does	Example	Result
`ROW_NUMBER()`	Unique sequence 1,2,3...	`ROW_NUMBER() OVER (ORDER BY Salary DESC)`	1,2,3,4,5
`RANK()`	Same rank for ties, then skips	`RANK() OVER (ORDER BY Salary DESC)`	1,2,2,4
`DENSE_RANK()`	Same rank for ties, no skip	`DENSE_RANK() OVER (ORDER BY Salary DESC)`	1,2,2,3
`NTILE(n)`	Splits rows into n equal groups	`NTILE(4) OVER (ORDER BY Salary)`	1..4

Example: rank employees inside each department

```
SELECT Name, Dept, Salary,
       RANK() OVER (PARTITION BY Dept ORDER BY Salary DESC) AS DeptRank
FROM Employees;
```

Classic interview question: Find the 2nd highest salary.

```
SELECT DISTINCT Salary
FROM (SELECT Salary,
             DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
      FROM Employees) x
WHERE rnk = 2;
```

Running totals with aggregate window functions

```
SELECT Name, Salary,
       SUM(Salary) OVER (ORDER BY EmpID) AS RunningTotal,
       AVG(Salary) OVER (PARTITION BY Dept) AS DeptAvg
FROM Employees;
```

9. Analytic functions

These look at other rows relative to the current row. Perfect for comparing this month with last month.

Function	What it does	Example	Result
`LAG(col,n,def)`	Value from n rows before	`LAG(Salary) OVER (ORDER BY EmpID)`	previous row salary
`LEAD(col,n,def)`	Value from n rows after	`LEAD(Salary) OVER (ORDER BY EmpID)`	next row salary
`FIRST_VALUE(col)`	First value in the window	`FIRST_VALUE(Name) OVER (ORDER BY Salary DESC)`	highest earner
`LAST_VALUE(col)`	Last value in the window	`needs ROWS frame`	-
`PERCENT_RANK()`	Relative rank 0 to 1	`PERCENT_RANK() OVER (ORDER BY Salary)`	0..1
`CUME_DIST()`	Cumulative distribution	`CUME_DIST() OVER (ORDER BY Salary)`	0..1
`PERCENTILE_CONT(p)`	Median / percentile (interpolated)	`PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary) OVER ()`	median

Example: salary change versus previous employee

```
SELECT Name, Salary,
       LAG(Salary) OVER (ORDER BY EmpID)          AS PrevSalary,
       Salary - LAG(Salary) OVER (ORDER BY EmpID) AS Difference
FROM Employees;
```

10. Other useful functions

Function	What it does	Example	Result
`@@ROWCOUNT`	Rows affected by last statement	`SELECT @@ROWCOUNT`	number
`@@IDENTITY`	Last identity value (any scope)	`SELECT @@IDENTITY`	number
`SCOPE_IDENTITY()`	Last identity in current scope (safer)	`SELECT SCOPE_IDENTITY()`	number
`@@VERSION`	SQL Server version text	`SELECT @@VERSION`	version info
`DB_NAME()`	Current database name	`SELECT DB_NAME()`	e.g. Company
`SUSER_NAME()` / `USER_NAME()`	Login / user name	`SELECT SUSER_NAME()`	login
`HOST_NAME()`	Client computer name	`SELECT HOST_NAME()`	PC name
`OBJECT_ID(name)`	Internal id of an object	`SELECT OBJECT_ID('Employees')`	number
`COL_LENGTH(t,c)`	Defined size of a column	`SELECT COL_LENGTH('Employees','Name')`	bytes
`ISNUMERIC(x)`	1 if value looks numeric	`SELECT ISNUMERIC('12a')`	0
`ISJSON(s)`	1 if valid JSON (2016+)	`SELECT ISJSON('{"a":1}')`	1
`JSON_VALUE(j,path)`	Reads a value from JSON	`SELECT JSON_VALUE('{"a":5}','$.a')`	5
`OPENJSON(j)`	Turns JSON into rows	`SELECT * FROM OPENJSON(@j)`	rows
`HASHBYTES(alg,x)`	Hash value (SHA2_256, etc.)	`SELECT HASHBYTES('SHA2_256','abc')`	binary
`CHECKSUM(x)`	Quick numeric hash	`SELECT CHECKSUM('abc')`	number
`GENERATE_SERIES(a,b)`	Rows from a to b (2022+)	`SELECT value FROM GENERATE_SERIES(1,5)`	1..5

User-defined functions (your own)

When built-in functions are not enough, you can write your own.

```
CREATE FUNCTION dbo.GetBonus (@Salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Salary * 0.10;
END;
GO

SELECT Name, dbo.GetBonus(Salary) AS Bonus FROM Employees;
```

Types: **Scalar** (returns one value), **Inline table-valued** (returns a table from one SELECT, fastest) and **Multi-statement table-valued**.

11. Practice exercises

Use the Employees table from Section 1. Try each question yourself before checking the answers.

Q1. Show each employee name in capital letters and the length of the name.

Q2. Display the first 3 letters of every name.

Q3. Show how many years each employee has worked (use DATEDIFF).

Q4. Show names and salaries. Replace NULL salaries with 0.

Q5. Show the average salary for each department, only for departments with more than 1 employee.

Q6. Rank employees by salary within each department (highest first).

Q7. Show each employee and the salary of the previous employee (by EmpID).

Q8. Label salaries: 'High' if 55000 or more, 'Medium' if 40000 or more, otherwise 'Low'.

Q9. Display the join date in dd-MMM-yyyy format.

Q10. Find the 2nd highest salary.

Answers

A1.

```
SELECT UPPER(Name), LEN(Name) FROM Employees;
```

A2.

```
SELECT LEFT(Name,3) FROM Employees;
```

A3.

```
SELECT Name, DATEDIFF(year, JoinDate, GETDATE()) AS Years FROM Employees;
```

A4.

```
SELECT Name, ISNULL(Salary,0) AS Salary FROM Employees;
```

A5.

```
SELECT Dept, AVG(Salary) FROM Employees
GROUP BY Dept HAVING COUNT(*) > 1;
```

A6.

```
SELECT Name, Dept, Salary,
  RANK() OVER (PARTITION BY Dept ORDER BY Salary DESC) AS rnk
FROM Employees;
```

A7.

```
SELECT Name, Salary,
  LAG(Salary) OVER (ORDER BY EmpID) AS PrevSalary
FROM Employees;
```

A8.

```
SELECT Name, Salary,
  CASE WHEN Salary >= 55000 THEN 'High'
       WHEN Salary >= 40000 THEN 'Medium'
       ELSE 'Low' END AS Grade
FROM Employees;
```

A9.

```
SELECT Name, FORMAT(JoinDate,'dd-MMM-yyyy') FROM Employees;
```

A10.

```
SELECT DISTINCT Salary FROM (
  SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) r
  FROM Employees) x WHERE r = 2;
```

Study tips for students

- Type every example yourself; do not just read.
- Change one value and predict the result before running.
- Learn one section per day and finish its exercises.
- Use the official Microsoft Learn documentation to see every parameter of a function.